

INDEX

<i>Program 1: Lowest Product Price.....</i>	<i>1</i>
<i>Program 2: Matrix Addition.....</i>	<i>3</i>
<i>Program 3: Add 2 Complex Numbers.....</i>	<i>6</i>
<i>Program 4: Symmetric Matrix.....</i>	<i>8</i>
<i>Program 5: Inner Class.....</i>	<i>10</i>
<i>Program 6: Search Element In Array.....</i>	<i>12</i>
<i>Program 7: String Manipulation.....</i>	<i>14</i>
<i>Program 8: Array Of Objects.....</i>	<i>16</i>
<i>Program 9: Sort Strings.....</i>	<i>18</i>
<i>Program 10: Function Overloading.....</i>	<i>20</i>
<i>Program 11: Single Inheritance.....</i>	<i>22</i>
<i>Program 12: Multilevel Inheritance.....</i>	<i>25</i>
<i>Program 13: Interface.....</i>	<i>29</i>
<i>Program 14: Package.....</i>	<i>33</i>
<i>Program 15: User Defined Exception.....</i>	<i>37</i>
<i>Program 16: User Defined Exception (Negative Numbers).....</i>	<i>41</i>
<i>Program 17: Thread Class.....</i>	<i>43</i>
<i>Program 18: Runnable Interface.....</i>	<i>46</i>
<i>Program 19: Generic Stack.....</i>	<i>49</i>
<i>Program 20: ArrayList.....</i>	<i>53</i>
<i>Program 21: Queue.....</i>	<i>57</i>
<i>Program 22: Set Using Linked Hashset.....</i>	<i>60</i>
<i>Program 23: Simple Calculator -AWT.....</i>	<i>63</i>

<i>Program 24: Mouse Events.....</i>	<i>67</i>
<i>Program 25: Subdirectory and Files.....</i>	<i>70</i>
<i>Program 26: Files: Write, Read and Display.....</i>	<i>73</i>
<i>Program 27: Copy File to Another.....</i>	<i>75</i>
<i>Program 28: Files: Even and Odd Numbers to Separate Files.....</i>	<i>77</i>

Date: 20/02/25

Program 1: Lowest Product Price

Aim: Define a class 'product' with data members pcode, pname and price. Create 3 objects of the class and find the product having the lowest price.

Algorithm:

1. Create a class named **product**. Declare instance variables **pcode**, **pname** and **price**.
2. Declare a method **details** that takes a integer **n** as a parameter.
 - Prompt the user to enter product details for the nth item.
 - Read the input values and assign them to the respective instance variables.
3. Declare a method **lowest** that takes two **product** objects as parameters(**a** and **b**).
 - Compare the price of the current product object (**this**), price of object **a**, and price of object **b**. Find the lowest price. (if this.price < (a.price and b.price) then this.price is the lowest, else if a.price<b.price then a.price is the lowest, else b.price is the lowest).
 - Display the product having lowest price.
4. Instantiate three objects of **product** class (**p1**, **p2**, **p3**).
5. Call the **details** method for each object.
6. Call the **lowest** method with **p1** object, passing **p2** and **p3** as parameters.

Source Code:

```
import java.util.Scanner;

class product
{
    String pcode,pname;
    int price;
    void details(int n)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("\nEnter product code for item "+n+": ");
        pcode=sc.next();
        System.out.print("Enter product name for item "+n+": ");
        pname=sc.next();
        System.out.print("Enter product price for item "+n+": ");
        price=sc.nextInt();
    }
    void lowest(product a,product b)
    {
        if(price<a.price && price<b.price)
```

```
        System.out.println("\nProduct "+pname+" has lowest price");
    else if(a.price<b.price)
        System.out.println("\nProduct "+a.pname+" has lowest price");
    else
        System.out.println("\nProduct "+b.pname+" has lowest price");
    }
}
class compare_objects
{
    public static void main(String[] args)
    {
        product p1=new product();
        product p2=new product();
        product p3=new product();
        p1.details(1);
        p2.details(2);
        p3.details(3);
        p1.lowest(p2, p3);
    }
}
```

Output:

```
Enter product code for item 1: 001
Enter product name for item 1: ABC
Enter product price for item 1: 1200

Enter product code for item 2: 002
Enter product name for item 2: XYZ
Enter product price for item 2: 2500

Enter product code for item 3: 003
Enter product name for item 3: IJK
Enter product price for item 3: 1000

Product IJK has lowest price
```

Result: The program has executed successfully and required output is obtained.

Date: 20/02/25

Program 2: Matrix Addition

Aim: Read 2 matrices from the console and perform matrix addition.

Algorithm:

1. Create class named **addmatrix**.
 - Declare instance variables **n** and **m** for the size of the matrix, a 2D array **matrix** to store the matrix elements.
2. Declare a **constructor** where user is prompted to enter the size of the matrix.
 - Initialize the matrix array with the specified size.
 - Prompt the user to enter the values of the matrix elements.
3. Declare a method named **add** that takes another **addmatrix** object **b** as a parameter.
 - Check if the sizes of the two matrices are compatible for addition (number of rows and columns match).
 - If not compatible, print a message indicating that addition is not possible.
 - If compatible, perform element-wise addition of the matrices and update the current matrix.
 - Display the sum matrix by calling the display method.
4. Declare a method named **display** to print the elements of a matrix.
 - Use nested loops to iterate through the rows and columns of the matrix.
 - Print each element of the matrix followed by a space.
 - Print a newline character after each row.
5. Instantiate two objects of the **addmatrix** class (**a** and **b**).
6. Call the **add** method of the first matrix (**a**), passing the second matrix (**b**) as a parameter.

Source Code:

```
import java.util.*;

class addmatrix
{
    Scanner sc=new Scanner (System.in);
    int n,m;
    int matrix[][];
    addmatrix(int x)
    {
        System.out.print("Enter size of matrix "+x+": ");
        n=sc.nextInt();
        m=sc.nextInt();
        matrix=new int[n][m];
        System.out.println("Enter value of matrix "+x+": ");
    }
}
```

```
        for(int i=0;i<n;i++)
            for(int j=0;j<m;j++)
                matrix[i][j]=sc.nextInt();
    }
    void add(addmatrix b)
    {
        if(n!=b.n || m!=b.m)
            System.out.println("Addition of matrix not possible");
        else
        {
            for(int i=0;i<n;i++)
                for(int j=0;j<m;j++)
                    matrix[i][j]+=b.matrix[i][j];
            System.out.println("Sum of matrix");
            this.display();
        }
    }
    void display()
    {
        for(int i=0;i<n;i++)
        {
            for(int j=0;j<m;j++)
                System.out.print(matrix[i][j]+" ");
            System.out.println();
        }
    }
}
class addmatrix_main
{
    public static void main(String[] args)
    {
        addmatrix a=new addmatrix(1);
        addmatrix b=new addmatrix(2);
        a.add(b);
    }
}
```

```
}  
}
```

Output:

```
Enter size of matrix 1: 3 3  
Enter value of matrix 1:  
1 2 3  
4 5 6  
7 8 9  
Enter size of matrix 2: 3 3  
Enter value of matrix 2:  
9 8 7  
6 5 4  
3 2 1  
Sum of matrix  
10 10 10  
10 10 10  
10 10 10
```

Result: The program has executed successfully and required output is obtained.

Date: 20/02/25

Program 3: Add 2 Complex Numbers

Aim: Read 2 complex numbers and perform addition on the 2 complex numbers.

Algorithm:

1. Create class named **complex**.
 - Declare instance variables **x** and **y** to represent the real and imaginary parts of a complex number.
2. Declare a method named **input** that takes an integer **n** as a parameter.
 - Prompt the user to enter the real and imaginary parts of the complex number for the **nth** instance.
 - Read the input values and assign them to the instance variables **x** and **y**.
3. Declare a method named **add** that takes another **complex** object **b** as a parameter.
 - Create a new **complex** object **c** to store the result of the addition.
 - Calculate the sum of the real parts (**x**) and imaginary parts (**y**) of the current object (**this**) and the object **b**.
 - Display the result of the addition in the form of complex numbers.
4. Instantiate two objects of the **complex** class (**a** and **b**).
5. Call the **input** method for each object to input the real and imaginary parts.
6. Call the **add** method of the first complex number (**a**), passing the second complex number (**b**) as a parameter.

Source Code:

```
import java.util.*;

class complex
{
    int x,y;

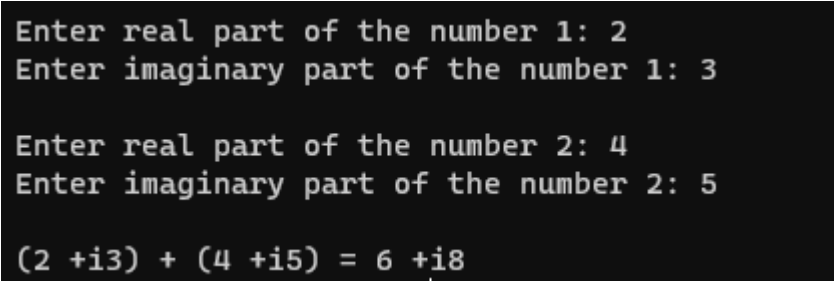
    Scanner sc=new Scanner(System.in);

    void input(int n)
    {
        System.out.print("\nEnter real part of the number "+n+": ");
        x=sc.nextInt();

        System.out.print("Enter imaginary part of the number "+n+": ");
        y=sc.nextInt();
    }

    void add(complex b)
    {
        complex c=new complex();
        c.x=x+b.x;
```

```
        c.y=y+b.y;
        System.out.println("\n("+x+" +i"+y+") + (" +b.x+" +i"+b.y+") = "+c.x+" +i"+c.y);
    }
}
class complex_main
{
    public static void main(String[] args)
    {
        complex a=new complex();
        complex b=new complex();
        a.input(1);
        b.input(2);
        a.add(b);
    }
}
```

Output:A screenshot of a terminal window with a black background and white text. It shows the execution of a Java program for complex number addition. The user is prompted to enter the real and imaginary parts of two numbers. The first number has a real part of 2 and an imaginary part of 3. The second number has a real part of 4 and an imaginary part of 5. The final output shows the sum: (2 +i3) + (4 +i5) = 6 +i8.

```
Enter real part of the number 1: 2
Enter imaginary part of the number 1: 3

Enter real part of the number 2: 4
Enter imaginary part of the number 2: 5

(2 +i3) + (4 +i5) = 6 +i8
```

Result: The program has executed successfully and required output is obtained.

Date: 27/02/25

Program 4: Symmetric Matrix

Aim: Read a matrix from the console and check whether it is symmetric or not.

Algorithm:

1. Create class named **symmetric**.
 - Declare instance variables **n** and **m** to represent the size of the matrix,
 - a 2D array **matrix** to store the matrix elements,
 - a variable **c** to track whether the matrix is symmetric (initialized to 0).
2. Define a constructor which prompts the user to enter the size of the matrix
 - Initialize the **matrix** array with the specified size.
 - Prompt the user to enter the values of the matrix elements.
3. Declare a method named **transpose** to check if the matrix is symmetric.
 - Use nested loops to iterate through the rows and columns of the matrix.
 - Check if the element at position (**i**, **j**) is equal to the element at position (**j**, **i**) for all **i** and **j**.
 - If any such pair of elements is not equal, set **c** to 1 and break out of the loop.
 - If **c** remains 0 after the loops, print that the matrix is symmetric; otherwise, print that it is not symmetric.
4. Instantiate an object of the **symmetric** class (**a**).
5. Call the **transpose** method to check if the matrix is symmetric.

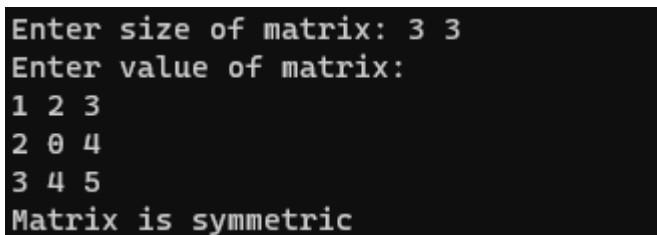
Source Code:

```
import java.util.*;

class symmetric
{
    Scanner sc=new Scanner (System.in);
    int n,m,matrix[][];
    int c=0;
    symmetric()
    {
        System.out.print("Enter size of matrix: ");
        n=sc.nextInt();
        m=sc.nextInt();
        matrix=new int[n][m];
        System.out.println("Enter value of matrix: ");
        for(int i=0;i<n;i++)
            for(int j=0;j<m;j++)
                matrix[i][j]=sc.nextInt();
    }
}
```

```
    }  
    void transpose()  
    {  
        for(int i=0;i<n;i++)  
        {  
            for(int j=0;j<m;j++)  
                if(matrix[j][i]!=matrix[i][j])  
                {  
                    c=1;  
                    break;  
                }  
        }  
        if(c!=1)  
            System.out.println("Matrix is symmetric");  
        else  
            System.out.println("Matrix is not symmetric");  
    }  
}  
class symmetric_main  
{  
    public static void main(String[] args)  
    {  
        symmetric a=new symmetric();  
        a.transpose();  
    }  
}
```

Output:



```
Enter size of matrix: 3 3  
Enter value of matrix:  
1 2 3  
2 0 4  
3 4 5  
Matrix is symmetric
```

Result: The program has executed successfully and required output is obtained.

Date: 27/02/25

Program 5: Inner Class

Aim: Create CPU with attribute price. Create inner class Processor (no. of cores, manufacturer) and static nested class RAM (memory, manufacturer). Create an object of CPU and print information of Processor and RAM.

Algorithm:

1. Create a class named **CPU**. Declare instance variables **price**.
2. Define a non-static inner class **Processor** within the **CPU** class:
 - Declare integer variable **no_cores** and string variable **manufacturer**.
 - In the constructor of this class, read and set **no_cores**, **manufacturer**.
3. Define a static inner class **RAM** within the **CPU** class:
 - Declare integer variable **memory** and string variable **manufacturer**.
 - In the constructor of this class, read and set **memory**, **manufacturer**.
4. Create an instance of the **CPU** class (**c**).
5. Create an instance of the non-static inner class **Processor** using the object of CPU class.
6. Create an instance of the static inner class **RAM**.
7. Calculate the CPU price based on the no: of cores in the processor and the RAM capacity.
8. Print the details of the processor and RAM using the information obtained from the user input.

Source Code:

```
import java .util.*;

class CPU
{
    Scanner sc= new Scanner(System.in);
    int price;
    class Processor
    {
        int no_cores;
        String manufacturer;
        Processor()
        {
            System.out.print("Enter no:of cores in the processor: ");
            no_cores =sc.nextInt();
            System.out.print("Enter manufacturer of the processor: ");
            manufacturer =sc.next();
        }
    }
}
```

```
static class RAM
{
    Scanner sc= new Scanner(System.in);
    int memory;
    String manufacturer;
    RAM()
    {
        System.out.print("Enter memory capacity of RAM: ");
        memory =sc.nextInt();
        System.out.print("Enter manufacturer of the RAM: ");
        manufacturer =sc.next();
    }
}

class innerclass
{
    public static void main(String[] args)
    {
        CPU c = new CPU();
        CPU.Processor p=c.new Processor();
        CPU.RAM r= new CPU.RAM();
        c.price=2500+(p.no_cores*20)+(r.memory*15);
        System.out.println("\n Processor:\nNo:of Cores : "+p.no_cores+"\nManufacturer:
        "+p.manufacturer);
        System.out.println("\n RAM:\nMemory Capacity : "+r.memory+" GB\nManufacturer:
        "+r.manufacturer);
        System.out.println("\nCPU Price: "+c.price);
    }
}
```

Output:

```
Enter no:of cores in the processor: 8
Enter manufacturer of the processor: Intel
Enter memory capacity of RAM: 16
Enter manufacturer of the RAM: Corsair

Processor:
No:of Cores : 8
Manufacturer: Intel

RAM:
Memory Capacity : 16 GB
Manufacturer: Corsair

CPU Price: 2900
```

Result: The program has executed successfully and required output is obtained.

Date: 06/03/25

Program 6: Search Element In Array

Aim: Program to search for an element in an array.

Algorithm:

1. Create a class named **array**. Declare instance variables **arr** (an integer array), **size** (to store the size of the array).
2. Define a constructor that takes an integer **n** as a parameter.
 - Initialize **size** with **n** and create an integer array **arr** of size **n**.
 - Prompt the user to enter the elements to array (**arr**).
3. Define a method named **search** that takes an integer **n** as a parameter.
4. Initialize a variable **c** to 0 to represent whether the number is found in the array.
5. Iterate through the array and check if the element at the current index is equal to **n**.
 - If found, set **c** to 1, print that the number is found, and break out of the loop.
 - If not found, print that the number is not an element of the array
6. In the **main** method, Prompt the user to enter the number of elements in the array (**n**).
 - Create an object **a** of the class **array**.
 - Prompt the user to enter the number to be searched (**num**).
 - Call the **search** method of the **array** object (**a**) with the specified number (**num**).

Source Code:

```
import java.util.*;

class array
{
    Scanner sc=new Scanner(System.in);
    int arr[],size;
    array(int n)
```

```
{
    size=n;
    arr=new int[n];
    System.out.println("Enter elements to the array: ");
    for(int i=0;i<n;i++)
        arr[i]=sc.nextInt();
}
void search(int n)
{
    int c=0;
    for(int i=0;i<size;i++)
        if(arr[i]==n)
        {
            c=1;
            System.out.println(n+" is found in the array.");
            break;
        }
    if(c==0)
        System.out.println(n+" is not an element of the array.");
}
}
class search_array
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter no:of elements in the array: ");
        int n=sc.nextInt();
        array a=new array(n);
        System.out.print("Enter number to be searched: ");
        int num=sc.nextInt();
        a.search(num);
    }
}
```

Output:

```
Enter no:of elements in the array: 5
Enter elements to the array:
10 20 30 40 50
Enter number to be searched: 20
20 is found in the array.
```

Result: The program has executed successfully and required output is obtained.

Date: 06/03/25

Program 7: String Manipulation

Aim: Java program to demonstrate string manipulation functions.

Algorithm:

1. Create a class named **word**. Declare instance variable **s** to store the input string.
2. Define a constructor for the **word** class.
 - Prompt the user to enter a string and store it in **s**.
3. Define a method named **str_functions** to perform various string manipulation operations:
 - **s.toLowerCase()**: lowercase version of the string.
 - **s.toUpperCase()**: uppercase version of the string.
 - **s.length()**: length of the string;
 - **s.substring(n)**: substring of the sting starting from index **n**.
 - **s.substring(n, m)**: substring of the string from index **n** to **m**.
 - **s.trim()**: trimmed version of the string.
 - **s.indexOf(char)**: index of the first occurrence of character **char**.
 - **s.indexOf(char, i)**: index of the first occurrence of character **char** starting from index **i**.
 - **s.concat(str2)**: concatenate string **str2** with string **s**.
4. In the **main** method:
 - Create an object **w** of the **word** class.
 - Call the **str_functions** method with object **w** to perform the string manipulation operations.

Source Code:

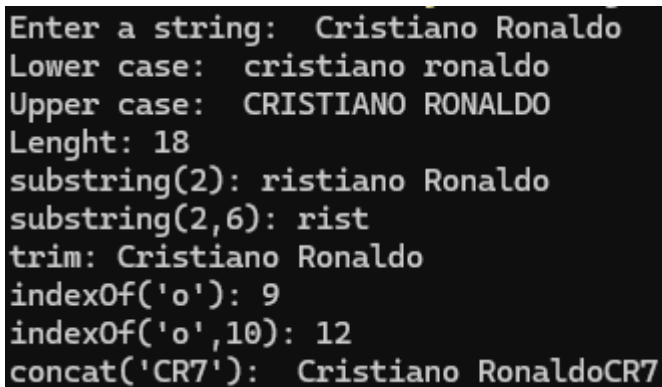
```
import java.util.*;

class word
{
    Scanner sc=new Scanner(System.in);
    String s;
    word()
    {
        System.out.print("Enter a string: ");
        s=sc.nextLine();
    }
    void str_functions()
    {
        System.out.println("Lower case: "+s.toLowerCase());
        System.out.println("Upper case: "+s.toUpperCase());
        System.out.println("Length: "+s.length());
    }
}
```

```
        System.out.println("substring(2): "+s.substring(2));
        System.out.println("substring(2,6): "+s.substring(2,6));
        System.out.println("trim: "+s.trim());
        System.out.println("indexOf('o'): "+s.indexOf('o'));
        System.out.println("indexOf('o',10): "+s.indexOf('o',10));
        System.out.println("concat('CR7'): "+s.concat("CR7"));
    }
}
```

```
class string_manipulation {
    public static void main(String[] args) {
        word w=new word();
        w.str_functions();
    }
}
```

Output:

A screenshot of a terminal window showing the output of a Java program. The text is as follows:

```
Enter a string: Cristiano Ronaldo
Lower case: cristiano ronaldo
Upper case: CRISTIANO RONALDO
Lenght: 18
substring(2): ristiano Ronaldo
substring(2,6): rist
trim: Cristiano Ronaldo
indexOf('o'): 9
indexOf('o',10): 12
concat('CR7'): Cristiano RonaldoCR7
```

Result: The program has executed successfully and required output is obtained.

Date: 06/03/25

Program 8: Array Of Objects

Aim: Program to create a class for Employee having attributes eNo, eName eSalary. Read n employ information and Search for an employee given eNo, using the concept of Array of Objects.

Algorithm:

1. Create a class named **Employee**. Declare instance variables for employee number (**eNo**), employee name (**eName**), and employee salary (**eSalary**).
2. Define a constructor to read and set **eNo**, **eName**, **eSalary**.
3. Prompt the user to enter the number of employees (**n**).
4. Create an array of **Employee** objects (**e[]**) with a size of **n**.
5. Use a **for** loop to iterate through each element of the array:
 - Initialize each element of the array by calling the **Employee** constructor
6. Prompt the user to enter an employee number to search (**num**).
7. Use a **for** loop to search for the employee with the specified number:
 - If found, print the employee name and salary.
 - If not found, print "Employee Not Registered!!!".

Source Code:

```
import java.util.*;

class Employee
{
    Scanner sc=new Scanner(System.in);
    int eNo;
    String eName;
    double eSalary;
    Employee(int n)
    {
        System.out.println("\t\t\tEnter Details for Employee "+n);
        System.out.print("Enter Employee Number: ");
        eNo=sc.nextInt();
        System.out.print("Enter Employee Name: ");
        eName=sc.next();
        System.out.print("Enter Salary: ");
        eSalary=sc.nextDouble();
    }
}

class array_objects
```

```
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter number of employees: ");
        int n=sc.nextInt();
        Employee e[]=new Employee[n];
        for(int i=0; i<n; i++)
        {
            e[i]=new Employee(i+1);
        }
        System.out.print("\nEnter Employee Number to search: ");
        int num=sc.nextInt();
        int c=0;
        for(int i=0; i<n; i++)
        {
            if(e[i].eNo==num)
            {
                System.out.println("Employee Name: "+e[i].eName+ "\nSalary: "+e[i].eSalary);
                c=1;
            }
        }
        if(c==0)
            System.out.println("Employee Not Registered!!!");
    }
}
```

Output:

```
Enter number of employees: 3
Enter Details for Employee 1
Enter Employee Number: 007
Enter Employee Name: M S Dhoni
Enter Salary: 1234.56
Enter Details for Employee 2
Enter Employee Number: 0028
Enter Employee Name: Ronaldo
Enter Salary: 7891.23
Enter Details for Employee 3
Enter Employee Number: 0010
Enter Employee Name: Messi
Enter Salary: 4567.89

Enter Employee Number to search: 007
Employee Name: M S Dhoni
Salary: 1234.56
```

Result: The program has executed successfully and required output is obtained.

Date: 13/03/25

Program 9: Sort Strings

Aim: Write a Java program to sort strings.

Algorithm:

1. Create a class named **sort_string**. Declare instance variables **s[]** (array of stringd) and **size**.
2. Define a constructor to read and **size** and **strings**.
3. Prompt the user to enter the number of strings (**size**), and store the stings into the array.
4. Define a method **sort()**:
 - Outer loop iterates over each elements of the array.
 - Inner loop iterates over the unsorted portion of the array.
 - Compare adjacent elements of the array and swap then if they are in the wrong order.
 - Repeat the process until the entire array is sorted.
5. Define method **display()** to display the array elements.
6. In the main method, create an object of the **sort_string class**.
7. Display original array.
8. Sort the array using **sort()**.
9. Display the sorted string array.

Source Code:

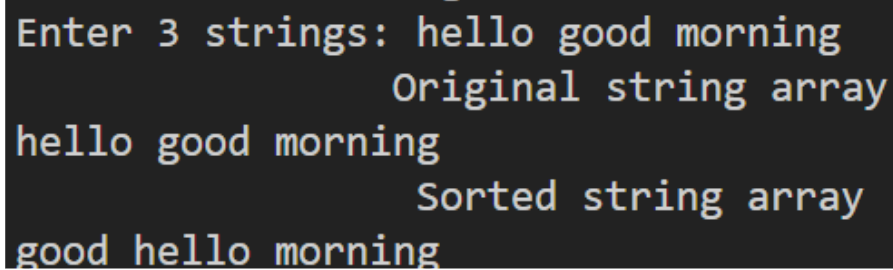
```
import java.util.*;

class sort_string
{
    Scanner sc=new Scanner(System.in);
    String s[];
    int size;
```

```
sort_string()
{
    System.out.print("Enter no: of strings: ");
    size=sc.nextInt();
    s=new String[size];
    System.out.print("Enter "+size+" strings: ");
    for(int i=0;i<size;i++)
        s[i]=sc.next();
}
void sort()
{
    for(int i=0;i<size;i++)
        for(int j=0;j<size-i-1;j++)
            if (s[j].compareTo(s[j+1])>0)
            {
                String temp=s[j];
                s[j]=s[j+1];
                s[j+1]=temp;
            }
}
void display()
{
    for(int i=0;i<size;i++)
        System.out.print(s[i]+" ");
}
}

public class sort_string_main
{
    public static void main(String[] args)
    {
        sort_string s1=new sort_string();
    }
}
```

```
        System.out.println("\t\tOriginal string array");
        s1.display();
        s1.sort();
        System.out.println("\n\t\tSorted string array");
        s1.display();
    }
}
```

Output:A screenshot of a terminal window showing the output of a Java program. The text is as follows:
Enter 3 strings: hello good morning
Original string array
hello good morning
Sorted string array
good hello morning

Result: The program has executed successfully and required output is obtained.

Date: 13/03/25

Program 10: Function Overloading

Aim: Program to calculate area of different shapes using overloaded functions.

Algorithm:

1. Create a class named **shape_area**, with the following methods.
 - **void area(double r):** Calculates and prints the area of a circle using the formula: $\pi * r * r$.
 - **void area(float r):** Calculates and prints the area of a square using the formula: side * side.
 - **void area(double l, double b):** Calculates and prints the area of a rectangle using the formula: length * breadth.
 - **void area(float b, float h):** Calculates and prints the area of a triangle using the formula: $0.5 * \text{base} * \text{height}$.
2. In the **main** method:
 - Create an object of class **shape_area**.
 - Prompt the user to enter the radius of the circle and call the **area(double r)** method with the provided radius.
 - Prompt the user to enter the length of the side of the square and call the **area(float r)** method with the provided side length.
 - Prompt the user to enter the length and breadth of the rectangle and call the **area(double l, double b)** method with these values.
 - Prompt the user to enter the base and height of the triangle and call the **area(float b, float h)** method with these values.

Source Code:

```
import java.util.*;

class shape_area
{
    void area(double r)
    {
        System.out.println("Area of circle: "+String.format("%.2f", 3.14*r*r));
    }
    void area(float r)
    {
        System.out.println("Area of square: "+(r*r));
    }
    void area(double l,double b)
    {
        System.out.println("Area of rectangle: "+(l*b));
    }
}
```

```
    }  
    void area(float b,float h)  
    {  
        System.out.println("Area of triangle: "+(0.5*b*h));  
    }  
  
}  
class area_main  
{  
    public static void main(String[] args)  
    {  
        Scanner sc=new Scanner(System.in);  
        shape_area s =new shape_area();  
        System.out.print("Enter radius of circle: ");  
        s.area(sc.nextDouble();  
  
        System.out.print("Enter length of side of square: ");  
        s.area(sc.nextFloat();  
  
        System.out.print("Enter lenght and breadth of rectangle: ");  
        double a=sc.nextDouble();  
        double b=sc.nextDouble();  
        s.area(a,b);
```

```
Enter radius of circle: 3  
Area of circle: 28.26  
Enter length of side of square: 5  
Area of square: 25.0  
Enter lenght and breadth of rectangle: 3 6  
Area of rectangle: 18.0  
Enter breadth and height of triangle: 4 8  
Area of triangle: 16.0
```

Output:

Result: The program has executed successfully and required output is obtained.
Date: 20/03/25

Program 11: Single Inheritance

Aim: Create a class 'Employee' with data members Empid, Name, Salary, Address and constructors to initialize the data members. Create another class 'Teacher' that inherit the properties of class employee and contain its own data members department, Subjects taught and constructors to initialize these data members and also include display function to display all the data members. Use array of objects to display details of N teachers.

Algorithm:

1. Create a class named **Employee**.
 - Declare instance variables **Empid**, **name**, **address**, and **salary**.
 - Define a constructor that initializes the employee details.
2. Define a class **Teacher** inherited from **Employee**.
 - Declare additional instance variable **dept**, **subject[]**, and **no**(number of subjects taught).
 - Define a constructor that initializes teacher details by calling superclass constructors and then initialize the instance variable of class **Teacher**.
 - Define a method **display()** to display the details of the teacher.
3. In the **main** method:
 - Prompt the user to enter number of teachers (**n**).
 - Create an array **t[]** of type **Teacher** with size **n**.
 - Using a loop instantiate each **Teacher** object, prompting the user to enter details for each teacher.
 - Use another loop to display the details of each teacher by calling the **display()** method.

Source Code:

```
import java.util.*;

class Employee
{
    Scanner sc=new Scanner(System.in);
    int Empid;
    String name,address;
```

```
double salary;
Employee(int x)
{
    System.out.println("\\n\\t\\tEnter Details of Teacher "+x);
    System.out.print("Enter Employee Id: ");
    Empid=sc.nextInt();
    System.out.print("Enter Employee Name: ");
    name=sc.next();
    System.out.print("Enter Salary: ");
    salary=sc.nextDouble();
    System.out.print("Enter Address: ");
    address=sc.next();
}
}
```

```
class Teacher extends Employee
{
    Scanner sc=new Scanner(System.in);
    String dept,subject[];
    int no;
    Teacher(int x)
    {
        super(x);
        System.out.print("Enter Department: ");
        dept=sc.next();
        System.out.print("Enter no:of subjects taught: ");
        no=sc.nextInt();
        subject=new String[no];
        System.err.print("Enter list of subjects: ");
        for(int i=0;i<no;i++)
            subject[i]=sc.next();
    }
    void display(int j)
    {
```

```
System.out.println("\n\t\tTeacher "+j+" Details");
System.out.println("Employee Id: "+Empid);
System.out.println("Employee Name: "+name);
System.out.println("Salary: "+salary);
System.out.println("Address: "+address);
System.out.println("Department: "+dept);
System.out.print("Subjects: ");
```

```
Enter no:of teachers: 2

      Enter Details of Teacher 1
Enter Employee Id: 101
Enter Employee Name: ABC
Enter Salary: 25000
Enter Address: House No:123
Enter Department: MCA
Enter no:of subjects taught: 2
Enter list of subjects: OOPS DBMS

      Enter Details of Teacher 2
Enter Employee Id: 102
Enter Employee Name: XYZ
Enter Salary: 20000
Enter Address: House No:897
Enter Department: MCA
Enter no:of subjects taught: 1
Enter list of subjects: NSA

      Teacher 1 Details
Employee Id: 101
Employee Name: ABC
Salary: 25000.0
Address: House
Department: MCA
Subjects: OOPS DBMS

      Teacher 2 Details
Employee Id: 102
Employee Name: XYZ
Salary: 20000.0
Address: House
Department: MCA
Subjects: NSA
```

Result: The program has executed successfully and required output is obtained.

Date: 20/03/25

Program 12: **Multilevel Inheritance**

Aim: Create a class 'Person' with data members Name, Gender, Address, Age and a constructor to initialize the data members and another class 'Employee' that inherits the properties of class Person and also contains its own data members like Empid, Company_name, Qualification, Salary and its own constructor. Create another class 'Teacher' that inherits the properties of class Employee and contains its own data members like Subject, Department, Teacherid and also contain constructors and methods to display the data members. Use array of objects to display details of N teachers.

Algorithm:

1. Create a class named **Person**.

- Declare instance variables **name, address, gender, and age.**
 - Define a constructor to initialize the instance variables.
2. Declare a class **Employee** inherited from **Person**.
 - Declare instance variables **EmpId, CompanyName, qualification, and salary.**
 - Define a constructor to call the superclass constructor and initialize employee details.
 3. Declare a class **Teacher** inherited from **Employee**.
 - Declare instance variables **subject, department, and TeacherID.**
 - Define a constructor to call the superclass constructor and initialize teacher details.
 - Define a **display()** method to display the teacher details.
 4. In the **main** method:
 - Prompt the user to enter number of teachers (**n**).
 - Create an array **t[]** of type **Teacher** with size **n**.
 - Using a loop instantiate each **Teacher** object, prompting the user to enter details for each teacher.
 - Use another loop to display the details of each teacher by calling the **display()** method.

Source Code:

```
import java.util.*;

class Person
{
    Scanner sc=new Scanner(System.in);
    String name,address;
    char gender;
    int age;
    Person(int x)
    {
        System.out.println("\n\n\t\tEnter details of Teacher "+x);
        System.out.print("Enter Name: ");
        name=sc.next();
        System.out.print("Enter Gender(M/F/O): ");
        gender=sc.next().charAt(0);
        System.out.print("Enter Age: ");
        age=sc.nextInt();
        System.out.print("Enter Address: ");
        address=sc.next();
    }
}
```

```
class Employee extends Person
{
    Scanner sc=new Scanner(System.in);
    int EmpId;
    String CompanyName,qualification;
    double salary;
    Employee(int x)
    {
        super(x);
        System.out.print("Enter Employee ID: ");
        EmpId=sc.nextInt();
        System.out.print("Enter Company Name: ");
        CompanyName=sc.next();
        System.out.print("Enter Qualification: ");
        qualification=sc.next();
        System.out.print("Enter Salary: ");
        salary=sc.nextDouble();
    }
}
```

```
class Teacher extends Employee
{
    Scanner sc=new Scanner(System.in);
    String subject,department;
    int TeacherId;
    Teacher(int x)
    {
        super(x);
        System.out.print("Enter Teacher ID: ");
        TeacherId=sc.nextInt();
        System.out.print("Enter Subject: ");
        subject=sc.next();
        System.out.print("Enter Department: ");
    }
}
```

```
        department=sc.next();
    }
    void display(int n)
    {
        System.out.println("\\n\\n\\t\\tTeacher "+n+" Details");
        System.out.println("Name: "+name);
        System.out.println("Gender: "+gender);
        System.out.println("Address: "+address);
        System.out.println("Age: "+age);
        System.out.println("Employee ID: "+EmpId);
        System.out.println("Teacher ID: "+name);
        System.out.println("Company Name: "+CompanyName);
        System.out.println("Department: "+name);
        System.out.println("Qualification: "+qualification);
        System.out.println("Salary: "+salary);
        System.out.println("Subject: "+subject);

    }
}

class teacher_main
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter no:of teachers: ");
        int n =sc.nextInt();
        Teacher t[]=new Teacher[n];
        for(int i=0;i<n;i++)
            t[i]=new Teacher(i+1);
        for(int i=0;i<n;i++)
            t[i].display(i+1);
    }
}
```

```
}
```

Output:

```
Enter no:of teachers: 2

      Enter details of Teacher 1
Enter Name: ABC
Enter Gender(M/F/O): F
Enter Age: 25
Enter Address: House No 256
Enter Employee ID: 101
Enter Company Name: CET
Enter Qualification: PHD
Enter Salary: 35000
Enter Teacher ID: 335
Enter Subject: DBMS
Enter Department: MCA

      Enter details of Teacher 2
Enter Name: XYZ
Enter Gender(M/F/O): M
Enter Age: 26
Enter Address: House No: 876
Enter Employee ID: 102
Enter Company Name: CET
Enter Qualification: PHD
Enter Salary: 35000
Enter Teacher ID: 877
Enter Subject: OOPS
Enter Department: MCA
```

```
      Teacher 1 Details
Name: ABC
Gender: F
Address: House
Age: 25
Employee ID: 101
Teacher ID: ABC
Company Name: CET
Department: ABC
Qualification: PHD
Salary: 35000.0
Subject: DBMS

      Teacher 2 Details
Name: XYZ
Gender: M
Address: House
Age: 26
Employee ID: 102
Teacher ID: XYZ
Company Name: CET
Department: XYZ
Qualification: PHD
Salary: 35000.0
Subject: OOPS
```

Result: The program has executed successfully and required output is obtained.

Date: 20/03/25

Program 13: Interface

Aim: Create an interface having prototypes of functions `area()` and `perimeter()`. Create two classes Circle and Rectangle which implements the above interface. Create a menu driven program to find area and perimeter of objects.

Algorithm:

1. Define the **Shape** interface:
 - Declare two abstract methods **area()** and **perimeter()**.
2. Define the **Circle** class:
 - Declare a double variable **radius**.
 - Implement a constructor that takes the **radius** as a parameter and assigns it to the **radius** member variable.
 - Implement the **area()** method to calculate and print the area of the circle.
 - Implement the **perimeter()** method to calculate and print the perimeter of the circle.
3. Define the **Rectangle** class:
 - Declare double variables **length** and **breadth**.
 - Implement a constructor that takes the **length** and **breadth** as parameters and assigns them to the respective member variables.
 - Implement the **area()** method to calculate and print the area of the rectangle.
 - Implement the **perimeter()** method to calculate and print the perimeter of the rectangle.
4. In the **main** method:
 - Declare an integer variable **ch** to store the user's choice.
 - Start a **do-while loop** to display a menu and execute the corresponding choice until the user chooses to exit.
 - Inside the loop, display a menu for the user to choose between Circle, Rectangle, or Exit.
 - If the user chooses **Circle**:
 - Prompt the user to enter the **radius** of the circle.
 - Create a Circle object with the given radius.
 - Call the **area()** and **perimeter()** methods of the Circle object.
 - If the user chooses Rectangle:
 - Prompt the user to enter the **length** and **breadth** of the rectangle.
 - Create a Rectangle object with the given length and breadth.
 - Call the **area()** and **perimeter()** methods of the Rectangle object.

- If the user chooses Exit:
 - Print "Exiting..." and exit the loop.
- If the user enters an **invalid choice**, display "Invalid choice!".
- Repeat the loop until the user chooses to exit (option 3).

Source Code:

```
import java.util.*;

interface Shape
{
    public void area();
    public void perimeter();
}

class Circle implements Shape
{
    double radius;
    Circle(double r)
    {
        radius = r;
    }
    public void area()
    {
        System.out.println("Area of circle: " + String.format("%.2f", (3.14*radius*radius)));
    }
    public void perimeter()
    {
        System.out.println("Perimeter of circle: " + String.format( "%.2f", (2*3.14*radius)));
    }
}

class Rectangle implements Shape
{

```

```
double length,breadth;
Rectangle(double l,double b)
{
    length = l;
    breadth = b;
}
public void area()
{
    System.out.println("Area of rectangle: " + String.format("%.2f" ,(length*breadth)));
}
public void perimeter()
{
    System.out.println("Perimeter of rectangle: " + String.format( "%.2f",
        (2*(length+breadth))));
}
}
```

```
class circle_rect {
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        int ch;
        do
        {
            System.out.print("-----\n1.Circle\n2.Rectangle \n3.Exit\n-----\n
                nEnter your choice: ");
            ch=sc.nextInt();
            switch(ch)
            {
                case 1:
                    System.out.print("Enter radius of circle: ");
                    double r=sc.nextDouble();
```

```
        Circle c =new Circle(r);
        c.area();
        c.perimeter();
        break;
    case 2:
        System.out.print("Enter length and breadth of rectangle: ");
        double l=sc.nextDouble();
        double b=sc.nextDouble();
        Rectangle r1 =new Rectangle(l,b);
        r1.area();
        r1.perimeter();
        break;
    case 3:
        System.out.println("Exiting...");
        break;
    default:
        System.out.println("Invalid choice!");
    }
}while(ch!=3);
}
```

Output:

```
-----
1.Circle
2.Rectangle
3.Exit
-----
Enter your choice: 1
Enter radius of circle: 4
Area of circle: 50.24
Perimeter of circle: 25.12
-----
1.Circle
2.Rectangle
3.Exit
-----
Enter your choice: 2
Enter length and breadth of rectangle: 3 7
Area of rectangle: 21.00
Perimeter of rectangle: 20.00
-----
1.Circle
2.Rectangle
3.Exit
-----
Enter your choice: 3
Exiting...
```

Result: The program has executed successfully and required output is obtained.

Date: 9/04/24

Program 14: Package

Aim: Create a Graphics package that has classes and interfaces for figures Rectangle, Triangle, Square and Circle. Test the package by finding the area of these figures.

Algorithm:

1. Define the **Figure** interface, and declare an abstract method **area()**.
2. Define the **Circle** class:
 - Declare a double variable **radius**.
 - Implement a **constructor** that takes the **radius** as a parameter and assigns it to the **radius** member variable.
 - Implement the **area()** method to calculate and print the area of the circle.
3. Define the **Rectangle** class:
 - Declare double variables **length** and **breadth**.
 - Implement a **constructor** that takes the **length** and **breadth** as parameters and assigns them to the respective member variables.
 - Implement the **area()** method to calculate and print the area of the rectangle.
4. Define the **Square** class:
 - Declare a double variable **length**.
 - Implement a constructor that takes the **length** as a parameter and assigns it to the length member variable.
 - Implement the **area()** method to calculate and print the area of the square.
5. Define the **Triangle** class:
 - Declare double variables **breadth** and **height**.
 - Implement a constructor that takes the **breadth** and **height** as parameters and assigns them to the respective member variables.
 - Implement the **area()** method to calculate and print the area of the triangle.
6. In the main method:
 - Prompt the user to enter the **radius** of a **circle** and read the input.
 - Create a **Circle object** with the given radius.
 - Call the **area()** method of the Circle object to calculate and print the area of the circle.
 - Prompt the user to enter the **length** and **breadth** of a rectangle and read the inputs.
 - Create a **Rectangle object** with the given length and breadth.
 - Call the **area()** method of the Rectangle object to calculate and print the area of the rectangle.
 - Prompt the user to enter the **side of a square** and read the input.
 - Create a Square object with the given side length.
 - Call the **area()** method of the Square object to calculate and print the area of the square.
 - Prompt the user to enter the **height** and **breadth** of a triangle and read the inputs.
 - Create a Triangle object with the given height and breadth.
 - Call the **area()** method of the Triangle object to calculate and print the area of the triangle.

Source Code:

```
package Graphics;
```

```
interface Figure
```

```
{  
    public void area();  
}
```

```
public class Circle implements Figure
```

```
{  
    double radius;  
    public Circle(double r)  
    {  
        radius = r;  
    }  
    public void area()  
    {  
        System.out.println("Area of circle: " + String.format("%.2f" ,(3.14*radius*radius)));  
    }  
}
```

```
public class Rectangle implements Figure
```

```
{  
    double length;  
    double breadth;  
    public Rectangle(double l,double b)  
    {  
        length=l;  
        breadth=b;  
    }  
    public void area()  
    {
```

```
        System.out.println("Area of Rectangle: " + String.format("%.2f" ,(length*breadth)));
    }
}
```

public class **Square** implements **Figure**

```
{
    double length;
    public Square(double l)
    {
        length=l;
    }
    public void area()
    {
        System.out.println("Area of Square: " + String.format("%.2f" ,(length*length)));
    }
}
```

public class **Triangle** implements **Figure**

```
{
    double breadth;
    double height;
    public Triangle(double b,double h)
    {
        height=h;
        breadth=b;
    }
    public void area()
    {
        System.out.println("Area of Triangle: " + String.format("%.2f" ,(0.5*height*breadth)));
    }
}
```

Class Using Graphics Package

```
import java.util.Scanner;
import Graphics.*;

class Test_package {
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("\nEnter radius of circle: ");
        double cr=sc.nextDouble();
        Graphics.Circle c = new Graphics.Circle(cr);
        c.area();
        System.out.print("\nEnter length and breadth of rectangle: ");
        double l=sc.nextDouble();
        double b=sc.nextDouble();
        Graphics.Rectangle r = new Graphics.Rectangle(l,b);
        r.area();
        System.out.print("\nEnter side of square: ");
        l=sc.nextDouble();
        Graphics.Square s = new Graphics.Square(l);
        s.area();
        System.out.print("\nEnter height and breadth of triangle: ");
        l=sc.nextDouble();
        b=sc.nextDouble();
        Graphics.Triangle t = new Graphics.Triangle(b,l);
        t.area();
    }
}
```

```
Enter radius of circle: 3
Area of circle: 28.26

Enter length and breadth of rectangle: 4 7
Area of Rectangle: 28.00

Enter side of square: 5
Area of Square: 25.00

Enter height and breadth of triangle: 3 7
Area of Triangle: 10.50
```

Output:

Result: The program has executed successfully and required output is obtained.

Date: 9/04/24

Program 15: User Defined Exception

Aim: Write a user defined exception class to authenticate the user name and password.

Algorithm:

1. Define two custom exceptions: **nameexception** and **passexception**, both extending **RuntimeException**, and displaying the message passed with the exception.
2. Define a class **user** with instance variables **name** and **password**.
3. Declare a constructor to initialize the variables.
4. Declare a method **login** within **user** class to check if the provided username and password match the stored values. If they match, print **Login Successful**, otherwise throw a exception indicating invalid username or password.
5. In the **main class**:
 - Prompt the user to enter a name and validate it character by character.
 - If the character is not an alphabet, throw a **nameexception** indicating invalid name.
 - Prompt the user to enter a password and validate it.
 - If the password length is less than 8 characters, throw a **passexception**, indicating minimum length of password.
 - If the password does not contain at least one digit, throw a **passexception**, indicating minimum requirement of 1 digit in the password.
6. Create a **user** object with the provided name and password.
7. Prompt for login username and password.
8. Call the **login** method of the **user** object.

Source Code:

```
import java.util.*;

class nameexception extends RuntimeException
{
    nameexception(String s)
    {
        super(s);
    }
}

class passexception extends RuntimeException
{
    passexception(String s)
    {
```

```
        super(s);
    }
}

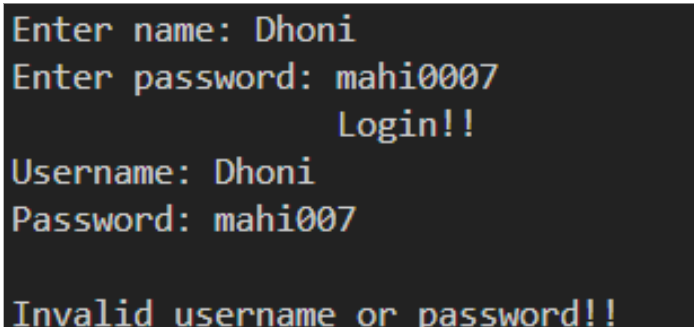
class user
{
    String name,password;
    user(String n, String p)
    {
        name=n;
        password=p;
    }
    void login(String n, String p)
    {
        try
        {
            if(name.equals(n)&&password.equals(p))
                System.out.println("\\t\\tLogin successful");
            else
                throw new passexception("Invalid username or password!! ");
        }
        catch (passexception e)
        {
            System.out.println("\\n"+e.getMessage());
            System.exit(0);
        }
    }
}

class validation
{
    public static void main(String[] args)
    {
```

```
Scanner sc=new Scanner(System.in);
System.out.print("Enter name: ");
String s=sc.next();
try
{
    for(int i=0;i<s.length();i++)
    {
        char ch=s.charAt(i);
        if((ch>=65&&ch<=90)||ch>=97&&ch<=122))
            continue;
        else
            throw new nameexception("Invalid Name");
    }
}
catch (nameexception e)
{
    System.out.println("\n"+e.getMessage());
    System.exit(0);
}
System.out.print("Enter password: ");
String pass=sc.next();
try
{
    int p=0;
    if(pass.length()<8)
        throw new nameexception("Password must have 8 characters ");
    for(int i=0;i<pass.length();i++)
    {
        char ch=pass.charAt(i);
        if((ch>=48&&ch<=57))
        {
            p=1;
        }
    }
}
```

```
        if(p==0)
            throw new passexception("Password must contain atleast 1 number");
    }
    catch (nameexception e)
    {
        System.out.println("\n"+e.getMessage());
        System.exit(0);
    }
    catch (passexception e)
    {
        System.out.println("\n"+e.getMessage());
        System.exit(0);
    }
    user u1=new user(s,pass);
    System.out.println("\t\tLogin!!");
    System.out.print("Username: ");
    String n1=sc.next();
    System.out.print("Password: ");
    String p1=sc.next();
    u1.login(n1,p1);
}
}
```

Output:



```
Enter name: Dhoni
Enter password: mahi0007
          Login!!
Username: Dhoni
Password: mahi007
Invalid username or password!!
```

```
Enter name: Dhoni  
Enter password: mahi  
Password must have 8 characters
```

Result: The program has executed successfully and required output is obtained.

Date: 9/04/24

Program 16: User Defined Exception (Negative Numbers)

Aim: Find the average of N positive integers, raising a user defined exception for each negative input.

Algorithm:

1. Define a custom exception class **negative** which extends **RuntimeException**. This exception is thrown when a negative number is encountered.
2. Define the main class **Average**.
3. Prompt the user to enter the number of elements **n**.
4. Create an integer array **nos** of size **n**.
5. Initialize a variable **sum** to store the sum of the entered numbers.
6. Prompt the user to enter **n** numbers and store them in the array **nos**.
 - While taking input, if any number entered is negative, throw a **negative** exception indicating that a negative number is not allowed.
 - If the entered number is positive, add it to the **sum**.
7. Calculate and display the average by dividing the **sum** by the number of elements (**n**).

Source Code:

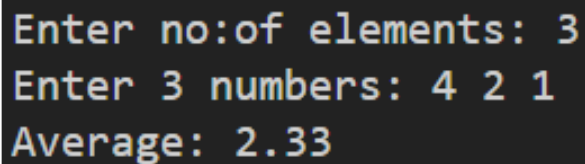
```
import java.util.*;

class negative extends RuntimeException
{
    negative(String s)
    {
        super(s);
    }
}

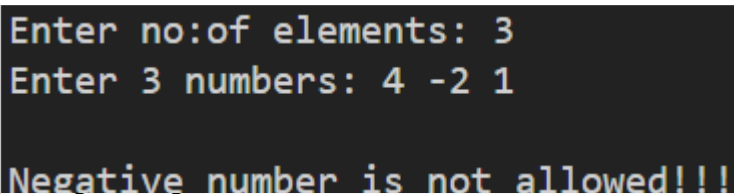
class Average
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter no:of elements: ");
        int n=sc.nextInt();
        int nos[]=new int[n];
        double sum=0;
```

```
System.out.print("Enter "+n+" numbers: ");  
for(int i=0;i<n;i++)  
{  
    nos[i]=sc.nextInt();  
    try  
    {  
        if(nos[i]<0)  
            throw new negative("Negative number is not allowed !!!");  
        else  
            sum+=nos[i];  
    }  
    catch(negative e)  
    {  
        System.out.println("\\n"+e.getMessage());  
        System.exit(0);  
    }  
}  
System.out.println("Average: " + String.format("%.2f", sum/n));  
  
}  
}
```

Output:



```
Enter no:of elements: 3  
Enter 3 numbers: 4 2 1  
Average: 2.33
```



```
Enter no:of elements: 3  
Enter 3 numbers: 4 -2 1  
Negative number is not allowed!!!
```

Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 17: Thread Class

Aim: Define 2 classes; one for generating multiplication table of 5 and other for displaying first N prime numbers. Implement using thread class.

Algorithm:

1. Define a class named **multiple5** that **extends** the **Thread** class.
2. Override the **run()** method in the **multiple5** class.
 - Iterate from 1 to 10.
 - For each iteration, print the value of **i multiplied by 5**.
3. Define another class named **Prime** that **extends** the **Thread** class.
4. Declare an integer variable **n** in the **Prime** class.
5. Define a **parameterized constructor** in the **Prime** class that takes an integer parameter limit and assigns it to the **n** variable.
6. Define a method named **isprime(int n)** in the **Prime** class that checks whether a given number n is prime or not.
 - Iterate from 2 to n/2.
 - If n is divisible by any number in this range, **return 0** (indicating not prime).
 - Otherwise, **return 1** (indicating prime).
7. Override the **run()** method in the **Prime** class.
 - Inside the **run()** method, iterate from 1 to n.
 - For each iteration, check if the number is prime using the **isprime()** method.
 - If it's prime, print the prime number.
8. In the **main()** method:
 - Create an instance **m** of the **multiple5** class.
 - Prompt the user to enter a **limit** for generating prime numbers.
 - Create an instance **m1** of the **Prime** class with the input limit n.
 - **Start** both threads **m** and **m1**.

Source Code:

```
import java.util.*;

class multiple5 extends Thread
{
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.println(i+" x 5 = "+(i*5));
        }
    }
}
```

```
class Prime extends Thread
{
    int n;
    Prime(int limit)
    {
        n=limit;
    }
    int isprime(int n)
    {
        for(int i=2;i<=n/2;i++)
        {
            if(n%i==0)
                return 0;
        }
        return 1;
    }
    public void run()
    {
        for(int i=1;i<=n;i++)
        {
            if(isprime(i)==1)
                System.out.println("Prime: "+i);
        }
    }
}

public class Threadclass {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        multiple5 m =new multiple5();
        System.out.print("Enter limit for generating prime: ");
        int n=sc.nextInt();
        Prime m1 =new Prime(n);
        m.start();
        m1.start();
    }
}
```

```
}  
}
```

Output:

```
Prime: 1  
1 x 5 = 5  
2 x 5 = 10  
3 x 5 = 15  
Prime: 2  
4 x 5 = 20  
Prime: 3  
5 x 5 = 25  
Prime: 5  
6 x 5 = 30  
Prime: 7  
7 x 5 = 35  
Prime: 11  
8 x 5 = 40  
Prime: 13  
9 x 5 = 45  
10 x 5 = 50
```

Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 18: Runnable Interface

Aim: Define 2 classes; one for generating Fibonacci numbers and other for displaying even numbers in a given range. Implement using threads. (Runnable Interface)

Algorithm:

1. Define a class named **fibonacci** that **implements** the **Runnable interface**.
2. Declare an integer variable **n** to store the limit.
3. Define a **parameterized constructor** in the **fibonacci** class that takes an integer parameter limit and assigns it to the **n** variable.
4. Implement the **run()** method in the fibonacci class.
 - Initialize variables **a** and **b** to **0** and **1** respectively.
 - Iterate from 1 to n.
 - **Print** the value of **a**.
 - Calculate the next Fibonacci number c by adding a and b.
 - Update a and b for the next iteration.
5. Define another class named **Even** that **implements** the **Runnable interface**.
6. Declare integer variables **start** and **end** in the Even class.
7. Define a **parameterized constructor** in the Even class that takes two integers **a** and **b** and assigns them to **start** and **end** respectively.
8. Implement the **run()** method in the Even class.
 - Iterate from start to end.
 - Check if the current number is even ($i \% 2 == 0$).
 - If it's even, print the even number.
9. In the main():
10. Prompt the user to enter the number of Fibonacci numbers (n).
11. Prompt the user to enter the start and end range for even numbers.
12. Create two **threads f** and **e** with **instances** of **fibonacci** and **Even** classes respectively, passing necessary parameters.
13. **Start** both threads **f** and **e**.

Source Code:

```
import java.util.*;

class fibonacci implements Runnable
{
    int n;

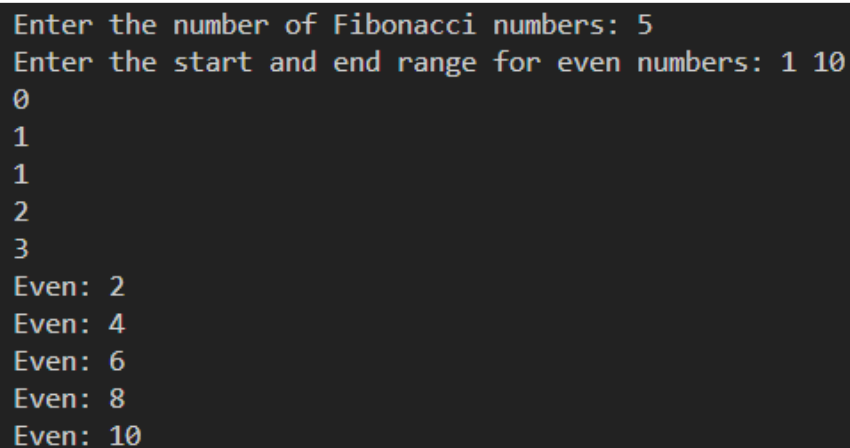
    fibonacci(int limit)
    {
        n=limit;
    }

    public void run()
    {
```

```
int a = 0, b = 1, c;  
for (int i = 1; i <= n; i++)  
{  
    System.out.println(a);  
    c = a + b;  
    a = b;  
    b = c;  
}  
}  
}
```

```
class Even implements Runnable  
{  
    int start,end;  
    Even(int a,int b)  
    {  
        start=a;  
        end=b;  
    }  
    public void run()  
    {  
        for (int i = start; i <= end; i++)  
        {  
            if (i % 2 == 0)  
            {  
                System.out.println("Even: "+i);  
            }  
        }  
    }  
}  
  
public class Runnable_interface  
{  
    public static void main(String[] args)  
    {
```

```
Scanner sc=new Scanner(System.in);
System.out.print("Enter the number of Fibonacci numbers: ");
int n = sc.nextInt();
System.out.print("Enter the start and end range for even numbers: ");
int start = sc.nextInt();
int end = sc.nextInt();
Thread f=new Thread(new fibonacci(n));
Thread e=new Thread(new Even(start,end));
f.start();
e.start();
}
}
```

Output:

```
Enter the number of Fibonacci numbers: 5
Enter the start and end range for even numbers: 1 10
0
1
1
2
3
Even: 2
Even: 4
Even: 6
Even: 8
Even: 10
```

Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 19: Generic Stack

Aim: Program to create a generic stack and do the Push and Pop operations.

Algorithm:

1. Define a generic class called stack with the following attributes:
 - **A:** An **ArrayList** to store elements of generic type T.
 - **top:** An integer representing the **index** of the **top** element in the stack. Initialized to -1.
 - **size:** An integer representing the **maximum size** of the stack.
2. Define a constructor that initializes the size of the stack and creates an ArrayList of the specified size.
3. Implement the **push** method:
 - Check if the stack is **full** ($\text{top} + 1 == \text{size}$).
 - If not, **increment top by 1** and add the element to the ArrayList at the top index.
4. Implement the **top** method:
 - Check if the stack is **empty** ($\text{top} == -1$).
 - If not, **return the element at the top** index of the ArrayList.
5. Implement the **pop** method:
 - Check if the stack is empty ($\text{top} == -1$).
 - If not, **decrement top by 1**.
6. Implement the empty method:
 - Return true if the stack is empty ($\text{top} == -1$), otherwise return false.
7. Implement the **toString** method to represent the stack as a string:
 - **Initialize an empty string.**
 - Iterate over the elements in the ArrayList from index 0 to top - 1.
 - Append each element to the string followed by '->'.
 - Append the last element (at index top) to the string.
 - Return the constructed string.
8. Define the **main** method:
 - Prompt the user to enter the **maximum size** of the stack and read the input.
 - Create an **instance** of the **stack** class with the specified maximum size.
 - Use a loop to push elements into the stack:
 - Print the stack after pushing all elements.
 - Pop an element from the stack.
 - Print the stack after popping.

Source Code:

```
import java.util.*;

class stack<T>
{
    ArrayList<T> A;
    int top = -1;
```

```
int size;
stack(int size)
{
    this.size = size;
    this.A = new ArrayList<T>(size);
}
void push(T X)
{
    if (top + 1 == size)
    {
        System.out.println("Stack Overflow");
    }
    else
    {
        top = top + 1;
        if (A.size() > top)
            A.set(top, X);
        else
            A.add(X);
    }
}
T top()
{
    if (top == -1)
    {
        System.out.println("Stack Underflow");
        return null;
    }
    else
        return A.get(top);
}
void pop()
{
    if (top == -1)
```

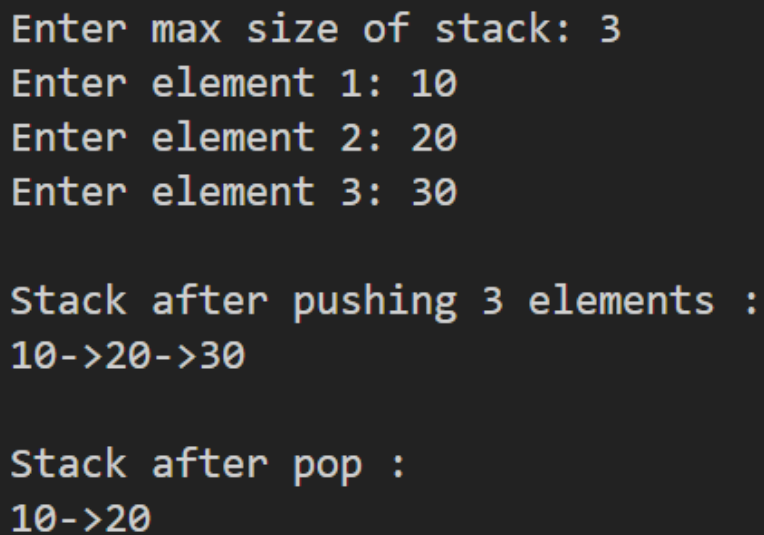
```
        {
            System.out.println("Stack Underflow");
        }
        else
            top--;
    }
    boolean empty() { return top == -1; }

    public String toString()
    {

        String Ans = "";
        for (int i = 0; i < top; i++)
        {
            Ans += String.valueOf(A.get(i)) + "->";
        }
        Ans += String.valueOf(A.get(top));
        return Ans;
    }
}

public class GenericStack {
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter max size of stack: ");
        int n=sc.nextInt();
        stack<Integer> s1 = new stack<>(n);
        int v;
        for(int i=0; i<n; i++)
        {
            System.out.print("Enter element "+(i+1)+" : ");
            v=sc.nextInt();
            s1.push(v);
        }
    }
}
```

```
        System.out.println("\nStack after pushing "+n+" elements :\n" + s1);
        s1.pop();
        System.out.println("\nStack after pop :\n" + s1);
    }
}
```

Output:A screenshot of a terminal window with a dark background and light-colored text. The text shows the execution of a program that manages a stack. It starts by asking for the maximum size of the stack (3), then for three elements (10, 20, 30). It then displays the stack after pushing these elements as '10->20->30'. Finally, it shows the stack after a pop operation as '10->20'.

```
Enter max size of stack: 3
Enter element 1: 10
Enter element 2: 20
Enter element 3: 30

Stack after pushing 3 elements :
10->20->30

Stack after pop :
10->20
```

Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 20: ArrayList

Aim: Program to maintain a list of Strings using ArrayList from collection framework, perform built-in operations.

Algorithm:

1. Create an empty **ArrayList** of Strings called **list**.
2. Create variables **el** (to store user input for elements) and **ch** (to store user input for choices).
3. Start a do-while loop with the condition **ch** not equal to 0.
4. Inside the loop, display a menu of options and prompt the user to enter their choice.
5. On the basis user's choice:
 - Case 1: Prompt the user to enter an element and add it to the **ArrayList** using **list.add(el)**.
 - Case 2: Print the size of the **ArrayList** using **list.size()**.
 - Case 3: Prompt the user to enter an **index** and print the element at that index using **list.get(index)**.
 - Case 4: Prompt the user to enter an **element** and **find its index** using **list.indexOf(el)**.
 - Case 5: Prompt the user to enter an **element** and check **if it exists** in the ArrayList using **list.contains(el)**.
 - Case 6: Prompt the user to enter an **element** and **remove** it from the ArrayList using **list.remove(el)**.
 - Case 7: Prompt the user to enter an **index** and **remove** the element at that index using **list.remove(index)**.
 - Case 8: **Print** the **entire ArrayList** using '**list**'.
 - Case 9: **Clear** the **ArrayList** using **list.clear()**
 - Case 0: Exit the loop.

Source Code:

```
import java.util.*;

public class ArrayList {

    public static void main(String[] args) {

        ArrayList <String> list = new ArrayList<String>();

        Scanner sc=new Scanner(System.in);

        String el;

        int ch;

        do

        {

            System.out.print("\n-----\n1: add\n2: size\n3: search by index\n4: find index\

n5: contains\n6: remove\n7: remove by index\n8: display\n9: clear search\n0:

Exit\n-----\nEnter your choice: ");
```

```
ch=sc.nextInt();
switch(ch)
{
    case 1:
        System.out.print("Enter element to insert: ");
        el=sc.next();
        list.add(el);
        break;
    case 2:
        System.out.println("Size of the arraylist: "+list.size());
        break;
    case 3:
        System.out.print("Enter index of element to search: ");
        int index=sc.nextInt();
        System.out.println("Element at index "+index+" is "+list.get(index));
        break;
    case 4:
        System.out.print("Enter an element to find index: ");
        el=sc.next();
        index=list.indexOf(el);
        System.out.println("Index of "+el+" is "+index);
        break;
    case 5:
        System.out.print("Enter a element");
        el=sc.next();
        boolean contains=list.contains(el);
        System.out.println(el+" is in the list :"+contains);
        break;
    case 6:
        System.out.print("Enter the element to be removed: ");
        el=sc.next();
        boolean removed=list.remove(el);
        System.out.println("After removing "+el+" ArrayList : "+list);
        break;
```

```

        case 7:
            System.out.print("Enter the index to remove the element: ");
            index=sc.nextInt();
            list.remove(index);
            System.out.println("After removing the element, arraylist:"+list);
            break;
        case 8:
            System.out.println("Arraylist: "+list);
            break;
        case 9:
            list.clear();
            break;
        case 0:
            break;
    }
}while(ch!=0);
}
}

```

Output:

```

-----
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
-----
Enter your choice: 1
Enter element to insert: 1

```

```

-----
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
-----
Enter your choice: 1
Enter element to insert: 3

```

```

-----
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
-----
Enter your choice: 2
Size of the arraylist: 3

```

```

-----
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
-----
Enter your choice: 3
Enter index of element to search: 2
Element at index 2 is 3

```

```

-----
1: add
2: size
3: search by index
4: find index
5: contains
6: remove
7: remove by index
8: display
9: clear search
0: Exit
-----
Enter your choice: 4
Enter an element to find index: 2
Index of 2 is 1

```

```
-----  
1: add  
2: size  
3: search by index  
4: find index  
5: contains  
6: remove  
7: remove by index  
8: display  
9: clear search  
0: Exit  
-----
```

```
Enter your choice: 5  
Enter a element6  
6 is in the list :false
```

```
-----  
1: add  
2: size  
-----
```

```
-----  
1: add  
2: size  
3: search by index  
4: find index  
5: contains  
6: remove  
7: remove by index  
8: display  
9: clear search  
0: Exit  
-----
```

```
Enter your choice: 7  
Enter the index to remove the element: 1  
After removing the element, arraylist:[1]
```

```
-----  
1: add  
2: size  
3: search by index  
4: find index  
5: contains  
6: remove  
7: remove by index  
8: display  
9: clear search  
0: Exit  
-----
```

```
Enter your choice: 6  
Enter the element to be removed: 3  
After removing 3 ArrayList: [1, 2]
```

Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 21: Queue

Aim: Program to demonstrate the creation of queue object using the PriorityQueue class.

Algorithm:

1. Create a new **PriorityQueue** object called q to store strings.
2. Declare variables el and ch of type String and int respectively.
3. Based on the users choice
 - Case I: **Add** :
 - Prompt the user to enter the clement to insert and store it in the variable cl.
 - Call the add method on the q object, passing el as the argument.
 - Case 2: **Remove**
 - Prompt the user to enter the clement to remove and store it in the variable el.
 - Call the remove method on the q object, passing el as the argument .
 - Case 3: **Display**
 - Print the contents of the q priority queue.
 - Case 4: **Head**
 - Print the head of the q priority queue using the peek method.
 - Case 5: **Poll**
 - Remove and return the head of the q priority queue using the poll method.
 - Case 0: **Wrong choice**

Source Code:

```
import java.util.*;
public class Queue
{
    public static void main(String[] args)
    {
        PriorityQueue<String>q=new PriorityQueue<String>();
        Scanner sc=new Scanner(System.in);
        String el;
        int ch;
        do
        {
            System.out.print("\n-----\n1:add\n2:remove \n3:display\n4:head\n0:wrong
                                choice\n-----\nEnter your choice: ");

            ch=sc.nextInt();
            switch(ch)
```

```
{
    case 1:
        System.out.print("Enter element to insert:\n");
        el=sc.next();
        q.add(el);
        break;
    case 2:
        q. remove();
        break;
    case 3:
        System.out.println("Priority queue:"+q);
        break;
    case 4:
        System.out.println("Head of the queue:"+q.peek());
        break;
    case 9:break;
}
}while(ch!=0);
}
```

Output:

```
-----
1:add
2:remove
3:display
4:head
0:Exit
-----
Enter your choice: 1
Enter element to insert: 1
```

```
-----
1:add
2:remove
3:display
4:head
0:Exit
-----
Enter your choice: 1
Enter element to insert: 2
```

```
-----  
1:add  
2:remove  
3:display  
4:head  
0:Exit  
-----  
Enter your choice: 1  
E  
-----  
1:add  
2:remove  
3:display  
4:head  
0:Exit  
-----  
Enter your choice: 3  
Priority queue:[2, 3]
```

```
-----  
1:add  
2:remove  
3:display  
4:head  
0:Exit  
-----  
Enter your choice: 3  
Priority queue:[1, 2, 3]
```

```
-----  
1:add  
2:remove  
3:display  
4:head  
0:Exit  
-----  
Enter your choice: 2
```

```
-----  
1:add  
2:remove  
3:display  
4:head  
0:Exit  
-----  
Enter your choice: 4  
Head of the queue:2
```

Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 22: Set Using Linked Hashset

Aim: Program to demonstrate the creation of Set object using the LinkedHashSet class.

Algorithm:

1. Create a new **LinkedHashSet** object called set to store strings.
2. Declare variables **el** and **ch** of type String and int respectively.
3. Based on the users choice:
 - Case 1: **Add**
 - Prompt the user to enter the element to insert and store it in the variable el.
 - Call the add method on the set object, passing el as the argument.
 - Case 2: **Remove**
 - Prompt the user to enter the element to remove and store it in the variable el.
 - Call the remove method on the set object, passing el as the argument.
 - Case 3: **Display**
 - Print the contents of the set linked hash set.
 - Case 4: **Search**
 - Prompt the user to enter the element to search and store it in the variable el.
 - Call the contains method on the set object, passing el as the argument, and store the result in a boolean variable contains.
 - Print whether the set contains the element or not.
 - Case 0: **Exit**
 - Break out of the switch statement.

Source Code:

```
import java.util.*;

public class Hashset
{
    public static void main(String[] args)
    {
        Set <String> set =new LinkedHashSet<String>();
        Scanner sc=new Scanner(System.in);
        String el;
        int ch;
        do
        {
            System.out.print("\n-----\n1: Add\n2: Remove\n3: Display\n4: Search\n0:
                               Exit\n-----\nEnter your choice: ");
```

```
ch=sc.nextInt();
switch(ch)
{
    case 1:
        System.out.print("Enter element to insert: ");
        el=sc.nextInt();
        set.add(el);
        break;
    case 2:
        System.out.print("Enter element to remove: ");
        el=sc.nextInt();
        set.remove(el);
        break;
    case 3:
        System.out.println("Linked Hashset: "+set);
        break;
    case 4:
        System.out.print("Enter element to search: ");
        el=sc.nextInt();
        boolean contains=set.contains(el);
        System.out.println("Set contains "+el+" : "+contains);
        break;
    case 0:
        System.out.println("Exiting");
        break;
}
}while(ch!=0);
}
```

Output:

```
-----
1: Add
2: Remove
3: Display
4: Search
0: Exit
-----
Enter your choice: 1
Enter element to insert: 3
```

```
-----  
1: Add  
2: Remove  
3: Display  
4: Search  
0: Exit  
-----  
Enter your choice: 1  
Enter element to insert: 1
```

```
-----  
1: Add  
2: Remove  
3: Display  
4: Search  
0: Exit  
-----  
Enter your choice: 1  
Enter element to insert: 2
```

```
-----  
1: Add  
2: Remove  
3: Display  
4: Search  
0: Exit  
-----  
Enter your choice: 3  
Linked Hashset: [1, 2, 3]
```

```
-----  
1: Add  
2: Remove  
3: Display  
4: Search  
0: Exit  
-----  
Enter your choice: 2  
Enter element to remove: 2
```

```
-----  
1: Add  
2: Remove  
3: Display  
4: Search  
0: Exit  
-----  
Enter your choice: 4  
Enter element to search: 2  
Set contains 2 : false
```

Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 23: Simple Calculator -AWT

Aim: Implement a simple calculator using AWT components

Algorithm:

1. Import necessary packages: **java.awt.*** and **java.awt.event.***.
2. Declare the **SimpleCalculator** class, which **extends Frame** and **implements ActionListener**.
3. Define class variables:
 - **textField**: TextField for displaying input and result.
 - **numberButtons**: Array of Buttons for numbers 0-9.
 - **operationButtons**: Array of Buttons for operations (+, -, *, /).
 - **equalsButton**: Button for performing the calculation.
 - **clearButton**: Button for clearing the text field.
 - **num1, num2**: Doubles to store operands.
 - **result**: Double to store the result of the operation.
 - **operation**: Character to store the current operation.
4. Implement the **constructor** SimpleCalculator():
 - Set the **title** and **size** of the **frame**.
 - Set layout to BorderLayout.
 - Create and configure the textField, add it to the NORTH of the frame.
 - Create **buttonPanel** with **GridLayout(4, 4)**.
 - Create **numberButtons** for digits 0-9, add ActionListener, and add them to **buttonPanel**.
 - Create **operationButtons** for operations (+, -, *, /), add ActionListener, and add them to **buttonPanel**.
 - Create **equalsButton** with ActionListener and add it to **buttonPanel**.
 - Create **clearButton** with ActionListener and add it to **buttonPanel**.
 - Add **buttonPanel** to the CENTER of the frame.
 - Add window closing listener to exit the application.
5. Implement **actionPerformed(ActionEvent ae)** method:
6. Get the action command from the event.
7. If the command is a digit (0-9), append it to the text field.
8. If the command is 'C', clear the text field.
9. If the command is '=', perform the calculation based on the stored operation and display the result in the text field.
10. If the command is an operation (+, -, *, /), store the current value in num1, store the operation, and clear the text field.
11. In the main method:

- Create an instance of SimpleCalculator.
- Make the calculator visible.

Source Code:

```
import java.awt.*;
import java.awt.event.*;

public class SimpleCalculator extends Frame implements ActionListener
{
    private TextField textField;
    private Button[] numberButtons;
    private Button[] operationButtons;
    private Button equalsButton;
    private Button clearButton;
    private double num1, num2, result;
    private char operation;

    public SimpleCalculator()
    {
        setTitle("Simple Calculator");
        setSize(300, 400);
        setLayout(new BorderLayout());

        textField = new TextField();
        textField.setEditable(false);
        add(textField, BorderLayout.NORTH);

        Panel buttonPanel = new Panel();
        buttonPanel.setLayout(new GridLayout(4, 4));

        numberButtons = new Button[10];
```

```
for (int i = 0; i < 10; i++)
{
    numberButtons[i] = new Button(String.valueOf(i));
    numberButtons[i].addActionListener(this);
    buttonPanel.add(numberButtons[i]);
}

operationButtons = new Button[4];
operationButtons[0] = new Button("+");
operationButtons[1] = new Button("-");
operationButtons[2] = new Button("*");
operationButtons[3] = new Button("/");
for (int i = 0; i < 4; i++)
{
    operationButtons[i].addActionListener(this);
    buttonPanel.add(operationButtons[i]);
}

equalsButton = new Button("=");
equalsButton.addActionListener(this);
buttonPanel.add(equalsButton);

clearButton = new Button("C");
clearButton.addActionListener(this);
buttonPanel.add(clearButton);

add(buttonPanel, BorderLayout.CENTER);

addWindowListener(new WindowAdapter()
{
    public void windowClosing(WindowEvent windowEvent)
    {
        System.exit(0);
    }
}
```

```
    });  
}  
  
public void actionPerformed(ActionEvent ae)  
{  
    String command = ae.getActionCommand();  
    if (command.charAt(0) >= '0' && command.charAt(0) <= '9')  
        textField.setText(textField.getText() + command);  
    else if (command.charAt(0) == 'C')  
        textField.setText("");  
    else if (command.charAt(0) == '=')  
    {  
        num2 = Double.parseDouble(textField.getText());  
        switch (operation)  
        {  
            case '+':  
                result = num1 + num2;  
                break;  
            case '-':  
                result = num1 - num2;  
                break;  
            case '*':  
                result = num1 * num2;  
                break;  
            case '/':  
                if (num2 != 0)  
                    result = num1 / num2;  
                else  
                    textField.setText("Cannot divide by zero");  
                break;  
        }  
        textField.setText(String.valueOf(result));  
    }  
    else
```

```
{  
    num1 = Double.parseDouble(textField.getText());  
    operation = command.charAt(0);  
    textField.setText("");  
}  
}  
  
public static void main(String[] args)  
{  
    SimpleCalculator calculator = new SimpleCalculator();  
    calculator.setVisible(true);  
}  
}
```

Output:



Result: The program has executed successfully and required output is obtained.

Date: 10/04/24

Program 24: Mouse Events

Aim: Develop a program to handle all mouse events and window events.

Algorithm:

1. Import the required libraries: **javax.swing.*** and **java.awt.event.***.
2. Define a class named **MouseEventDemo** that extends **JFrame**.
3. Define the constructor:
 - Set the title of the window to "Mouse Events Demo".

- Set the size of the window to 400x300 pixels.
 - Set the default close operation to exit on close.
4. Add a **MouseListener** to the frame using **addMouseListener(new MouseAdapter() { ... })**.
 5. Override methods for the following mouse events:
 - **mouseClicked(MouseEvent e)**: Print the coordinates of the mouse when clicked.
 - **mousePressed(MouseEvent e)**: Print the coordinates of the mouse when pressed.
 - **mouseReleased(MouseEvent e)**: Print the coordinates of the mouse when released.
 - **mouseEntered(MouseEvent e)**: Print the coordinates of the mouse when entered into the frame.
 - **mouseExited(MouseEvent e)**: Print the coordinates of the mouse when exited from the frame.
 6. Add a **WindowListener** to the frame using **addWindowListener(new WindowAdapter() { ... })**.
 7. Override the **windowClosing(WindowEvent e)** method to print a message when the window is closed.
 8. In the **main** method:
 - Use **SwingUtilities.invokeLater()** to ensure that Swing components are created and accessed in the Event Dispatch Thread for thread safety.
 - Create an instance of **MouseEventDemo**.
 - Set the visibility of the frame to true.

Source Code:

```
import javax.swing.*;
import java.awt.event.*;

public class MouseEventDemo extends JFrame
{
    public MouseEventDemo()
    {
        setTitle("Mouse Events Demo");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        addMouseListener(new MouseAdapter()
        {
            public void mouseClicked(MouseEvent e)
            {
                System.out.println("Mouse clicked at (" + e.getX() + ", " + e.getY() + ")");
            }
        })
    }
}
```

```
public void mousePressed(MouseEvent e)
{
    System.out.println("Mouse pressed at (" + e.getX() + ", " + e.getY() + ")");
}

public void mouseReleased(MouseEvent e)
{
    System.out.println("Mouse released at (" + e.getX() + ", " + e.getY() + ")");
}

public void mouseEntered(MouseEvent e)
{
    System.out.println("Mouse entered at (" + e.getX() + ", " + e.getY() + ")");
}

public void mouseExited(MouseEvent e)
{
    System.out.println("Mouse exited at (" + e.getX() + ", " + e.getY() + ")");
}

});

addMouseListener(new MouseMotionAdapter()
{
    public void mouseDragged(MouseEvent e)
    {
        System.out.println("Mouse dragged to (" + e.getX() + ", " + e.getY() + ")");
    }

    public void mouseMoved(MouseEvent e)
    {
        System.out.println("Mouse moved to (" + e.getX() + ", " + e.getY() + ")");
    }
});
```

```
addWindowListener(new WindowAdapter()
{
    public void windowClosing(WindowEvent e)
    {
        System.out.println("Window closed");
    }
});

public static void main(String[] args)
{
    SwingUtilities.invokeLater(() ->
    {
        MouseEventDemo demo = new MouseEventDemo();
        demo.setVisible(true);
    });
}
```

Output:

```
Mouse entered at (209, 31)
Mouse moved to (209, 31)
Mouse moved to (209, 32)
Mouse pressed at (209, 32)
Mouse released at (209, 32)
Mouse clicked at (209, 32)
Mouse pressed at (209, 32)
Mouse released at (209, 32)
Mouse clicked at (209, 32)
Mouse moved to (209, 31)
Mouse exited at (209, 30)
Window closed
```

Result: The program has executed successfully and required output is obtained.

Date: 19/04/24

Program 25: Subdirectory and Files

Aim: Program to list the sub directories and files in a given directory and also search for a file name.

Algorithm:

1. Define class **DirectoryListing**.
2. Define static method **listFilesAndDirectories(directory: File):**
 - For each file in directory:
 - Print file name.
 - If file is a directory, call listFilesAndDirectories recursively.
3. Define static method **searchFile(directory: File, fileName: String):**
 - For each file in directory:
 - If file matches fileName, print its absolute path.
 - If file is a directory, call searchFile recursively.
 - If file is not found, print a message indicating so.
4. Define **main** method:
 - Read directory path from user input.
 - Create File object directory with directory path.
 - If directory exists and is a directory:
 - Print files and directories in specified directory.
 - Prompt user to enter file name to search.
 - Search for file in directory.
 - If directory does not exist or is not a directory, print message indicating invalid directory path.

Source Code:

```
import java.io.File;
import java.util.Scanner;

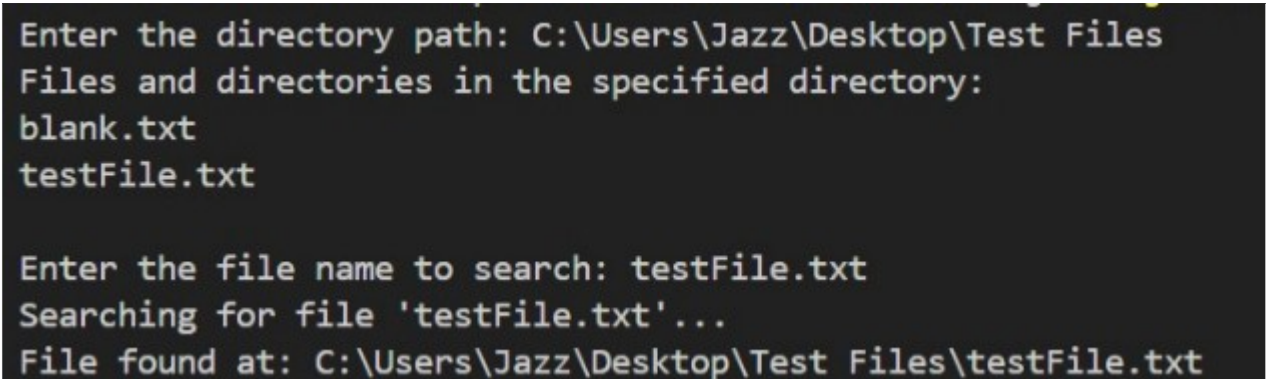
public class DirectoryListing
{
    static void listFilesAndDirectories(File directory)
    {
        File[] fileList = directory.listFiles();
        if (fileList != null)
        {
            for (File file : fileList)
            {
```

```
        System.out.println(file.getName());
        if (file.isDirectory())
            listFilesAndDirectories(file);
    }
}

static void searchFile(File directory, String fileName)
{
    File[] fileList = directory.listFiles();
    if (fileList != null)
    {
        for (File file : fileList)
        {
            if (file.isFile() && file.getName().equals(fileName))
            {
                System.out.println("File found at: " + file.getAbsolutePath());
                return;
            }
            else if (file.isDirectory())
                searchFile(file, fileName);
        }
    }
    System.out.println("File '" + fileName + "' not found.");
}

public static void main(String[] args)
{
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter the directory path: ");
    String directoryPath = scanner.nextLine();
    File directory = new File(directoryPath);
    if (directory.exists() && directory.isDirectory())
    {
```

```
        System.out.println("Files and directories in the specified directory:");
        listFilesAndDirectories(directory);
        System.out.print("\nEnter the file name to search: ");
        String fileName = scanner.nextLine();
        System.out.println("Searching for file '" + fileName + "'...");
        searchFile(directory, fileName);
    }
    else
        System.out.println("Invalid directory path.");
    scanner.close();
}
```

Output:A screenshot of a terminal window showing the execution of a Java program. The user enters the directory path 'C:\Users\Jazz\Desktop\Test Files'. The program lists the files 'blank.txt' and 'testFile.txt'. Then, the user enters the file name 'testFile.txt'. The program searches for it and outputs 'File found at: C:\Users\Jazz\Desktop\Test Files\testFile.txt'.

```
Enter the directory path: C:\Users\Jazz\Desktop\Test Files
Files and directories in the specified directory:
blank.txt
testFile.txt

Enter the file name to search: testFile.txt
Searching for file 'testFile.txt'...
File found at: C:\Users\Jazz\Desktop\Test Files\testFile.txt
```

Result: The program has executed successfully and required output is obtained.

Date: 19/04/24

Program 26: Files: Write and Read

Aim: Write a program to write to a file, then read from the file and display the contents on the console.

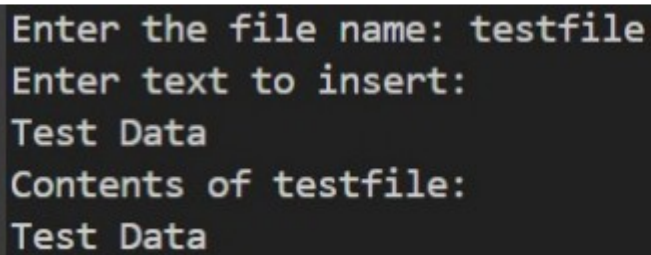
Algorithm:

1. Prompt the user to enter the file name and store it in **fileName**.
2. Create a **FileOutputStream (fos)** with **fileName**.
3. Prompt the user to enter text to insert and store it in **text**.
4. Write text to fos.
5. Close fos.
6. Create a **FileInputStream (fis)** with **fileName**.
7. Read bytes from **fis** into a byte array (**b**).
8. Close fis.
9. Convert byte array (**b**) to a string (**fileContent**).
10. Print "Contents of " + fileName + ":".
11. Print fileContent.

Source Code:

```
import java.io.*;
import java.util.*;
public class WriteRead
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the file name: ");
        String fileName = sc.nextLine();
        FileOutputStream fos = new FileOutputStream(fileName);
        System.out.println("Enter text to insert: ");
        String text = sc.nextLine();
        fos.write(text.getBytes());
        FileInputStream fis = new FileInputStream(fileName);
        byte[] b = new byte[fis.available()];
        fis.read(b);
        fis.close();
        String fileContent = new String(b);
```

```
        System.out.println("Contents of " + fileName + ":");  
        System.out.println(fileContent);  
        sc.close();  
    }  
}
```

Output:A screenshot of a terminal window with a dark background and light-colored text. The text shows the program's execution flow: a prompt to enter a file name, the user input 'testfile', a prompt to enter text to insert, the user input 'Test Data', a prompt to show the contents of the file, and the output 'Test Data'.

```
Enter the file name: testfile  
Enter text to insert:  
Test Data  
Contents of testfile:  
Test Data
```

Result: The program has executed successfully and required output is obtained.

Date: 19/04/24

Program 27: Copy File to Another

Aim: Write a program to copy one file to another.

Algorithm:

1. Create **BufferedReader** (reader) for user input.
2. Prompt user for source file path, store in **sourceFile**.
3. Prompt user for destination file path, store in **destinationFile**.
4. Open **input stream (in)** for source file, **output stream (out)** for destination file.
5. Read from input stream into buffer, write to output stream until end of file.
6. Close input and output streams.
7. Close **BufferedReader**.

Source Code:

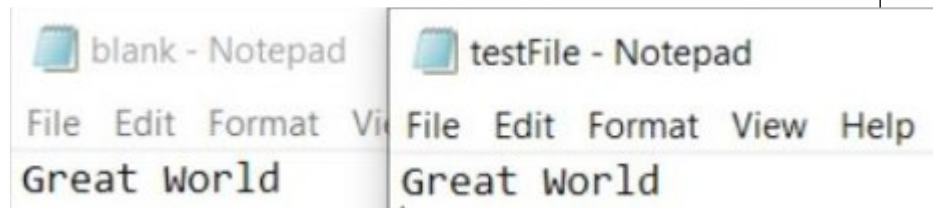
```
import java.io.*;

public class FileCopy
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader reader = new BufferedReader(new InputStreamReader (System.in));
        System.out.print("Enter the source file path: ");
        String sourceFile = reader.readLine();
        System.out.print("Enter the destination file path: ");
        String destinationFile = reader.readLine();
        try (InputStream in = new FileInputStream(sourceFile);
            OutputStream out = new FileOutputStream(destinationFile))
        {
            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = in.read(buffer)) != -1)
            {
                out.write(buffer, 0, bytesRead);
            }
            System.out.println("File copied successfully.");
        }
        catch (IOException e)
```

```
{  
    System.out.println("An error occurred during file copy: " + e.getMessage());  
}  
finally  
{  
    reader.close();  
}  
}
```

Output:

```
Enter the source file path: \Users\Jazz\Desktop\Test Files\testFile.txt  
Enter the destination file path: \Users\Jazz\Desktop\Test Files\blank.txt  
File copied successfully.
```



Result: The program has executed successfully and required output is obtained.

Date: 20/04/24

Program 28: Files: Even and Odd

Aim: Write a program that reads from a file having integers. Copy even numbers and odd numbers to separate files.

Algorithm:

1. Create a **BufferedReader** (reader) to read user input.
2. Prompt the user to enter the input file path and store it in **inputFile**.
3. Define file paths for **evenFile** and **oddFile**.
4. Create a **BufferedReader** (**fileReader**) to read from the input file.
5. Create a **BufferedWriter** (**evenWriter**) to write even numbers to the even file.
6. Create a **BufferedWriter** (**oddWriter**) to write odd numbers to the odd file.
7. Read each line from the input file, parse it to an integer, and determine if it's even or odd.
8. Write even numbers to the even file and odd numbers to the odd file.
9. Print a success message indicating the separation is complete.
10. Close the **BufferedReader**, **BufferedWriter** for even numbers, and **BufferedWriter** for odd numbers.
11. Close the **BufferedReader** for user input.

Source Code:

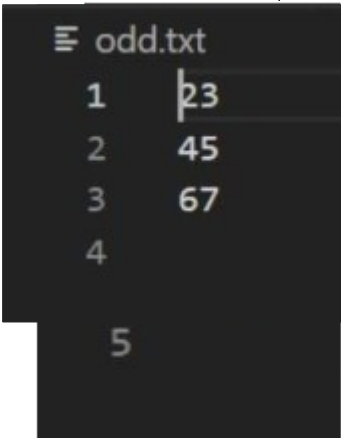
```
import java.io.*;

public class NumberSeparation
{
    public static void main(String[] args)
    {
        BufferedReader reader = new BufferedReader(new InputStreamReader (System.in));
        System.out.print("Enter the input file path: ");
        String inputFile = reader.readLine();
        String evenFile = "even.txt";
        String oddFile = "odd.txt";
        BufferedReader fileReader = new BufferedReader(new FileReader (inputFile));
        BufferedWriter evenWriter = new BufferedWriter(new FileWriter (evenFile));
        BufferedWriter oddWriter = new BufferedWriter(new FileWriter (oddFile));
        String line;
        while ((line = fileReader.readLine()) != null)
        {
```

```
int number = Integer.parseInt(line);
if (number % 2 == 0)
{
    evenWriter.write(line);
    evenWriter.newLine();
}
else
{
    oddWriter.write(line);
    oddWriter.newLine();
}
}
System.out.println("Even and odd numbers separated successfully. ");
reader.close();
fileReader.close();
evenWriter.close();
oddWriter.close();
}
}
```

Output:

```
Enter the input file path: C:\Users\Jazz\Desktop\Test Files\testFile.txt
Even and odd numbers separated successfully.
```



```
odd.txt
1 | 3
2
3
4
5
```

Result: The program has executed successfully and required output is obtained.

